

Fantasma da Máquina

- **Limite de tempo:** 10 minutos
- **Pontuação baseline:** 28.6
- **Ambiente:** uma GPU (≈ 16 GB VRAM), sem Internet
- **Tamanho da solução:** `solution.ipynb` ≤ 20 MB
- **Armazenamento:** 5 GB
- **Modelos pré-treinados:** apenas [bge-base-en-v1.5](#) — um **encoder** de texto (modelo de embedding).

Tarefa

Estão a acontecer coisas estranhas no Arquivo Nacional do Cazaquistão. Os bibliotecários dizem que alguns livros costumavam terminar de forma diferente, mas ninguém consegue prová-lo — todas as cópias são iguais e todas as histórias continuam a fazer sentido. É convidado, na qualidade de investigador de IA, a localizar estas alterações.



Uma passagem começa como texto escrito por um humano e, a certa altura, muda silenciosamente para uma continuação gerada por um modelo de linguagem. Quando esta passagem é lida como um todo, parece um texto coerente — mas, algures a meio, o autor mudou de uma pessoa para uma máquina. A sua tarefa é **encontrar essa transição: o índice do carácter onde termina a parte humana e começa a parte da máquina**.

Cada amostra é uma única string `text`. Existe exatamente uma fronteira. Tudo o que a precede é humano; tudo a partir dela é gerado por uma máquina.

Conjunto de dados

Passagens em inglês em texto simples, cada uma com uma fronteira.

- **Parte A** (antes da fronteira): um excerto de texto escrito por um humano.
- **Parte B** (a partir da fronteira): uma continuação produzida por um modelo de linguagem, condicionada pela Parte A.
- Cada parte tem pelo menos 180 palavras; o comprimento total é de ~500–800 palavras.
- O `boundary_char_index` é o índice em caracteres onde começa a Parte B: `text[:boundary_char_index]` é a parte humana, contendo o espaço que separa as duas partes, e `text[boundary_char_index:]` é a parte gerada pela máquina.

O que recebe

Recebe **duas pastas**:

Pasta	Amostras	answers.jsonl?	Utilize-a para
dataset/train/	1,221	❑ incluído	treinar / fazer o fine-tune do seu método
dataset/test_public/	380	❑ incluído (cópia de desenvolvimento)	executar a sua pipeline e calcular localmente a sua própria pontuação

No **momento da avaliação**, a sua pasta `dataset/test_public/` é **substituída por um conjunto de avaliação oculto**. Tendo o mesmo formato, mas **sem** `answers.jsonl`. O seu notebook é novamente executado sobre esse conjunto, e o `answers.jsonl` que produz é avaliado.

- A tabela classificativa pública utiliza um conjunto **test_leaderboard_a** oculto (380 amostras).
- A classificação final utiliza um conjunto **test_leaderboard_b** oculto (380 amostras).

Os três conjuntos de avaliação têm o mesmo tamanho e são extraídos da mesma distribuição que `train`, pelo que a sua pontuação `dataset/test_public/` local é uma estimativa razoável da sua pontuação na tabela classificativa final.

Formato em disco

```
dataset/train/data.jsonl      # one JSON object per line: {"id": "...", "text": "..."}
dataset/train/answers.jsonl  # {"id": "...", "boundary_char_index": 1842}
dataset/test_public/data.jsonl      # {"id": "...", "text": "..."}
dataset/test_public/answers.jsonl  # dev copy only – ABSENT in the hidden grading set
```

- Os IDs em `answers.jsonl` correspondem aos IDs em `data.jsonl`.
- `dataset/train/` (com respostas) está disponível sempre que efetuar treino ou ajuste fino.

Saída (formato de submissão)

Deve submeter **um único notebook, que tem de se chamar** `solution.ipynb`. Este nome de ficheiro exato é obrigatório. Qualquer outro será rejeitado sem ser executado.

O seu notebook tem de ler `dataset/test_public/data.jsonl` e escrever um único ficheiro `answers.jsonl` na raiz do repositório — um objeto JSON por linha, associando cada ID de amostra ao índice de carácter previsto para a fronteira:

```
{"id": "example_000123", "boundary_char_index": 1790}
{"id": "example_000124", "boundary_char_index": 2450}
```

- `boundary_char_index` tem de ser um **número inteiro em $[0, \text{len}(\text{text})]$** .
- Cada ID em `dataset/test_public/data.jsonl` deve aparecer exatamente uma vez. Uma amostra em falta em `answers.jsonl` (ou com um valor não inteiro / fora do intervalo) recebe uma pontuação de 0 para essa amostra.

Avaliação

Para cada amostra, seja p o índice previsto e t a fronteira verdadeira. A pontuação por amostra decresce exponencialmente com a distância em caracteres:

$$\text{score} = \exp\left(-\frac{|p - t|}{\tau}\right) \in (0, 1], \text{ where } \tau = 100.$$

Isto conduz ao seguinte comportamento da pontuação:

- **=1.0** — carácter exato da fronteira;
- **≈0.78** — desvio de 25 caracteres; - **≈0.61** — desvio de 50 caracteres;
- **≈0.37** — desvio de 100 caracteres;
- **≈0.01** — desvio de 500 caracteres.

A **pontuação final é a média** das pontuações por amostra em todas as amostras da partição (apresentada numa escala de 0–100). A métrica recompensa a proximidade, não apenas a exatidão.

Restrições

- **Ambiente:** uma GPU (≈16 GB VRAM), sem Internet no momento da avaliação — o modelo permitido (abaixo) já é fornecido. **Limite de tempo real: 10 minutos** para a execução completa — este tem de abranger qualquer treino / ajuste fino que efetue no momento de avaliação **e ainda** a inferência no conjunto de avaliação.
- **Modelo pré-treinado permitido** — esta lista é exaustiva; não podem ser utilizados outros pesos pré-treinados. É **previamente fornecido no ambiente** (carregue-o normalmente, por exemplo, `from_pretrained`; não há Internet no momento da avaliação):
 - **bge-base-en-v1.5** — um **encoder** de texto com 110M parâmetros (modelo de embedding). Produz embeddings de frases/passagens; não é um modelo de linguagem generativo. Pode utilizá-lo **tal como está (features congeladas) ou fazer o seu ajuste fino na partição train** (o ajuste fino completo é compatível com o limite de 16 GB / 10 minutos).
- As ferramentas clássicas / estatísticas não têm restrições: pode construir qualquer modelo baseado em features (por exemplo, classificadores ou regressores do scikit-learn) sobre features de embedding que calcule por si próprio. Os *pesos pré-treinados de deep learning* estão restringidos apenas à lista acima.